Análise e Complexidade de Algoritmos

Aula 5

Análise de Recursividade

Profª Drª Patricia Pereira

Ibmec
PROTAGONISTAS PARA O MUNDO

Objetivos

O principal objetivo é aprender a olhar para um algoritmo recursivo e transformar o código em uma expressão matemática.

Ibmec
PROTAGONISTAS PARA O MUNDO

Revisão: o que é recursividade?

Lembrando:

Uma função recursiva é uma função que chama a si mesma para resolver instâncias menores do mesmo problema.

Toda função recursiva deve possuir, em geral:

- **Caso base:** Condição que interrompe a recursão.
- **Caso recursivo:** Parte em que a função chama a si mesma com um problema menor.

O problema deve caminhar em direção ao caso base.

Ibmec
PROTAGONISTAS PARA O MUNDO

Exemplo: fatorial

Exemplo: função fatorial versão recursiva

```
def fatorial(n):
    if n == 0:
        return 1
    return n * fatorial(n - 1)
```

Caso base:

Caso recursivo:

Exemplo: fatorial

Como a função é executada?

```
fatorial(4)
      ↓
4 x fatorial(3)
      ↓
3 x fatorial(2)
      ↓
2 x fatorial(1)
      ↓
1 x fatorial(0)
      ↓
1
```

Quantas chamadas da função foram realizadas?

Para n=4

5 chamadas

Ibmec
PROTAGONISTAS PARA O MUNDO

Por que analisar algoritmos recursivos?

Em algoritmos iterativos, frequentemente conseguimos contar diretamente quantas vezes um laço é executado. Por exemplo:

```
for i in range(n):
    print(i)
```

O **print()** é executado aproximadamente ***n*** vezes.

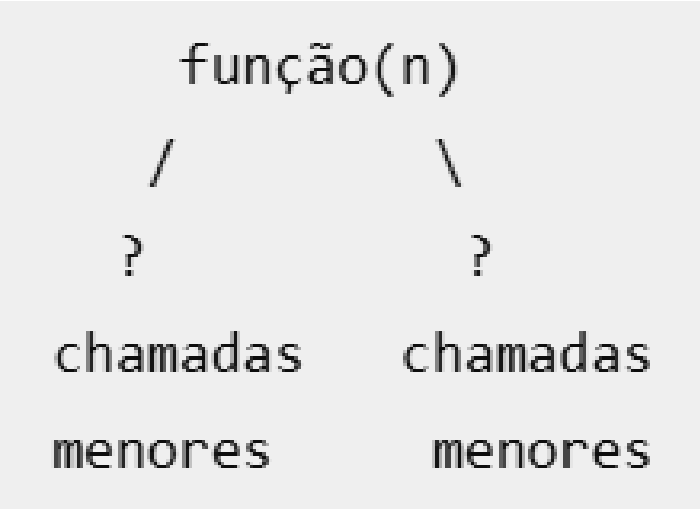
Logo: $T(n)=cn$, onde c é uma constante

Ou seja, $T(n)=\Theta(n)$

Ibmec
PROTAGONISTAS PARA O MUNDO

O problema da recursividade

Nos algoritmos recursivos, entretanto, uma chamada pode gerar **outras chamadas**, que geram novas chamadas.



Como podemos representar matematicamente essa estrutura?

equação de recorrência.

Ibmec
PROTAGONISTAS PARA O MUNDO

CONTAGEM DE PASSOS

O que é um passo elementar? **1 operação** $\approx c$

Exemplos

Podemos considerar como operações de custo constante:

- atribuição;
- comparação;
- operação aritmética;
- acesso a uma posição;
- chamada de função.

Não estamos interessados no valor exato de ***c***, mas em como o custo cresce quando ***n*** aumenta

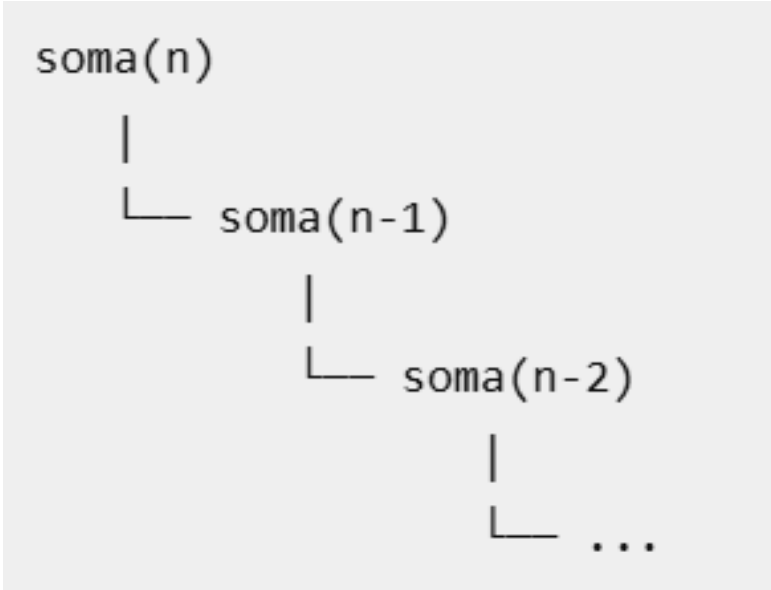
Contagem de passos elementares

Considerando:

```
def soma(n):
    if n == 1:
        return 1

    return n + soma(n - 1)
```

Vamos analisar, para n:



Ibmec
PROTAGONISTAS PARA O MUNDO

Contagem de passos elementares

Cada chamada realiza:

- comparação;
- operação de soma;
- chamada recursiva.

Vamos considerar esse conjunto como um custo constante: **c**

```
soma(n)
    |
    └── soma(n-1)
            |
            └── soma(n-2)
                    |
                    └── ...
```

$$T(n) = T(n-1) + c$$

$$T(1) = c_0$$

equação de recorrência

Ibmec
PROTAGONISTAS PARA O MUNDO

COMO CONSTRUIR UMA RECORRÊNCIA

Para construir uma recorrência, pergunte:

1. Quantas chamadas recursivas existem?
2. Qual o tamanho de cada subproblema?
3. Quanto trabalho é realizado fora da recursão?
4. Qual é o caso base?

Fórmula:

$$\boxed{T(n) = \text{chamadas recursivas} + \text{trabalho local}}$$

Ibmec
PROTAGONISTAS PARA O MUNDO

O que é uma equação de recorrência?

Uma recorrência expressa o custo de resolver um problema de tamanho ***n*** em função do custo de resolver problemas menores.

Forma geral:

$T(n) =$ **custo das chamadas recursivas** $+$ **custo do trabalho realizado**

Por exemplo:

$$T(n) = T(n-1) + c$$

Significa:
Para resolver um problema de tamanho ***n***, resolvemos um problema de tamanho (***n-1***) e gastamos mais um custo constante ***c***.

Ibmec
PROTAGONISTAS PARA O MUNDO

Exemplo 1 — Recursão linear

Considerando:

```
def exemplo(n):
    if n <= 1:
        return 1

    return exemplo(n - 1) + 1
```

Ibmec
PROTAGONISTAS PARA O MUNDO

Exemplo 1 — Recursão linear

Identificar a chamada recursiva

Existe uma chamada: exemplo(n - 1)

Portanto:

$$\boldsymbol{T(n) = T(n-1) + c}$$

$$T(1) = c_0$$

Ibmec
PROTAGONISTAS PARA O MUNDO

Exemplo 1 — Método da iteração: Recursão linear

Primeira expansão

$$T(n) = T(n-1) + c$$

$$T(n-1) = T(n-2) + c$$

$$T(n) = T(n-2) + c + c$$

$$\boxed{T(n) = T(n-2) + 2c}$$

Segunda expansão

$$T(n) = T(n-1) + c$$

$$T(n-2) = T(n-3) + c$$

$$T(n) = T(n-3) + c + c + c$$

$$\boxed{T(n) = T(n-3) + 3c}$$

Ibmec
PROTAGONISTAS PARA O MUNDO

Exemplo 1 — Recursão linear

Método da iteração

Depois de k expansões

$$T(n) = T(n - k) + kc$$

Exemplo 1 — Recursão linear

$$T(n) = T(n-k) + kc$$

A recursão termina quando: $n - k = 1$

Logo: $k = n - 1$

Então: $T(n) = T(1) + (n-1)c$

Portanto: $T(n) = \theta(n)$

Ibmec
PROTAGONISTAS PARA O MUNDO

Método da iteração — ideia geral

O método da iteração consiste em:

1. Escrever a recorrência;
2. Expandir a recorrência;
3. Identificar um padrão;
4. Generalizar;
5. Encontrar a condição de parada;
6. Determinar a complexidade.

Ibmec
PROTAGONISTAS PARA O MUNDO

Exemplo 2 — iteração com custo crescente

```
def soma_recursiva(n):
    if n <= 1:
        return 1

    for i in range(n):
        print(i)

    return soma_recursiva(n - 1)
```

Trabalho realizado na chamada atual

O for executa ***n*** vezes

Uma chamada recursiva

Portanto: $\boldsymbol{T(n) = T(n-1) + n}$

Com: $\boldsymbol{T(1) = c}$

Ibmec
PROTAGONISTAS PARA O MUNDO

Exemplo 2 — Aplicando a iteração

$$T(n) = T(n-1) + n$$

Expandindo: $T(n-1) = T(n-2) + (n-1)$

Logo: $T(n) = T(n-2) + (n-1) + n$

Expandindo novamente: $T(n-2) = T(n-3) + (n-2)$

Logo : $T(n) = T(n-3) + (n-2) + (n-1) + n$

Depois de k expansões:

$$T(n) = T(n-k) + (n-k+1) + \cdots + (n-1) + n$$

Ibmec
PROTAGONISTAS PARA O MUNDO

Exemplo 2 — Chegando ao caso base

A recursão termina quando: $\boldsymbol{n - k = 1}$

Logo: $\boldsymbol{k = n - 1}$

$$\boldsymbol{T(n) = T(n-k) + (n-k+1) + \cdots + (n-1) + n}$$

Então: $T(n) = T(1) + 2 + 3 + \cdots + n$

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

Portanto: $\boldsymbol{T(n) = \theta(n^2)}$

Ibmec
PROTAGONISTAS PARA O MUNDO

Exemplo 3 — analisando o algoritmo

```python
def processo(n):
    if n <= 1:
        return

    processo(n // 2)
    processo(n // 2)

    for i in range(n):
        print(i)
```

Como esse algoritmo está dividindo o problema?

- Quantas chamadas recursivas existem?
- Qual é o tamanho de cada problema gerado?
- Quanto trabalho é realizado fora da recursão?

Nosso objetivo é analisar o custo.

Ibmec
PROTAGONISTAS PARA O MUNDO

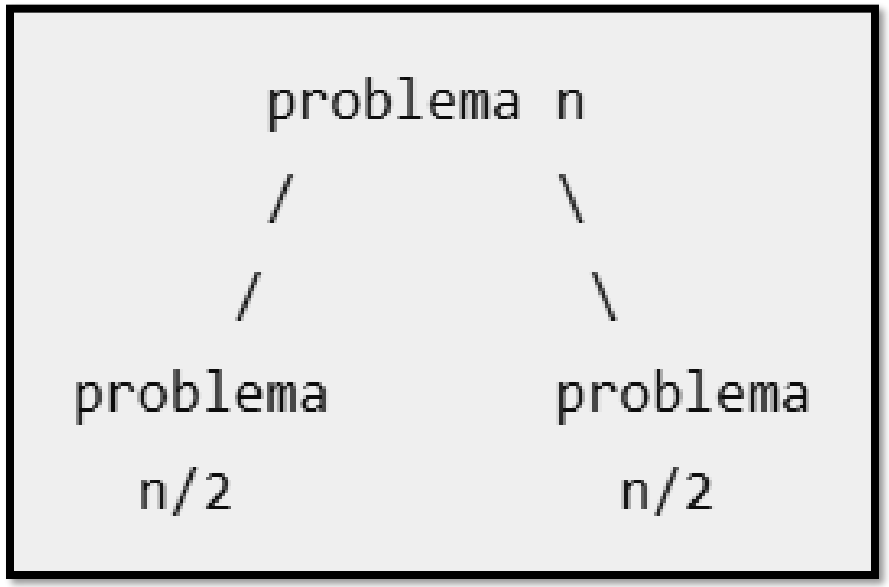
Exemplo 3 — analisando o algoritmo

```
def processo(n):
    if n <= 1:
        return

    processo(n // 2)
    processo(n // 2)

    for i in range(n):
        print(i)
```

Como esse algoritmo está dividindo o problema?



Ibmec
PROTAGONISTAS PARA O MUNDO

Exemplo 3 — analisando o algoritmo

```
def processo(n):
    if n <= 1:
        return

    processo(n // 2)
    processo(n // 2)

    for i in range(n):
        print(i)
```

tamanho do problema em cada chamada
n/2

2 chamadas recursivas

Trabalho realizado na chamada atual
O for executa ***n*** vezes

Ibmec
PROTAGONISTAS PARA O MUNDO

Exemplo 3 — analisando o algoritmo

```python
def processo(n):
    if n <= 1:
        return

    processo(n // 2)
    processo(n // 2)

    for i in range(n):
        print(i)
```

Como podemos transformar essas informações em uma equação de recorrência?

Chamadas recursivas: $\boldsymbol{2T(n/2)}$
Trabalho local: $\boldsymbol{n}$
Logo: $\boldsymbol{T(n) = 2T(n/2) + n}$

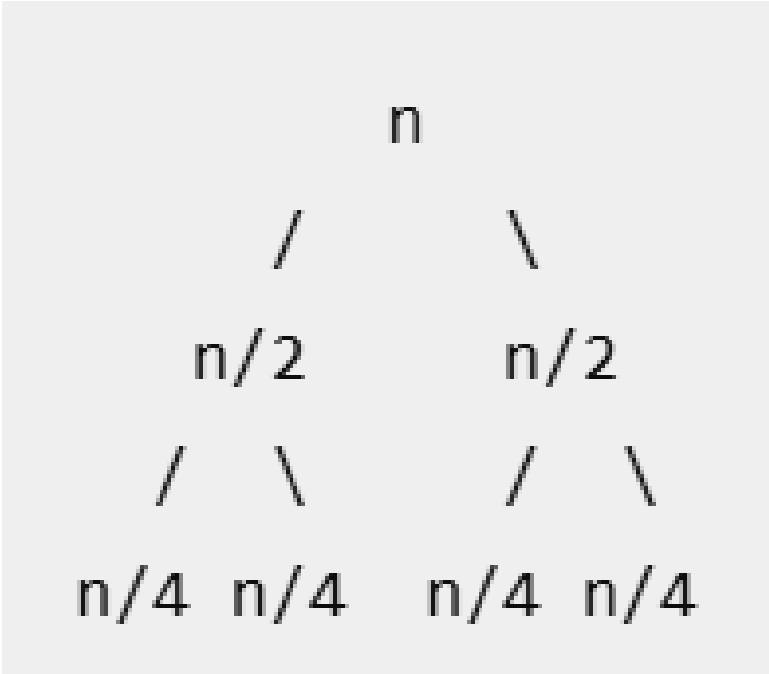
Agora temos a equação de recorrência que precisamos estudar a complexidade.

Ibmec
PROTAGONISTAS PARA O MUNDO

Exemplo 3 — ÁRVORE DE RECURSÃO

Antes de resolver: vamos enxergar a recorrência

$$T(n) = 2T(n/2) + n$$

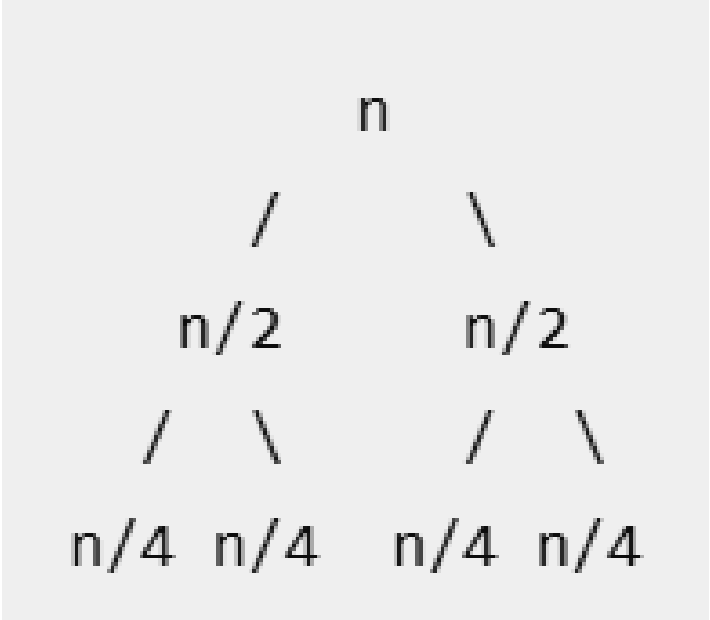


Cada nó representa uma chamada recursiva.

Ibmec
PROTAGONISTAS PARA O MUNDO

Exemplo 3 — ÁRVORE DE RECURSÃO

vamos enxergar a recorrência



Nível 0: $1 \times n = n$

Nível 1: $2 \times \frac{n}{2} = n$

Nível 2: $4 \times \frac{n}{4} = n$

Nível 3: $8 \times \frac{n}{8} = n$

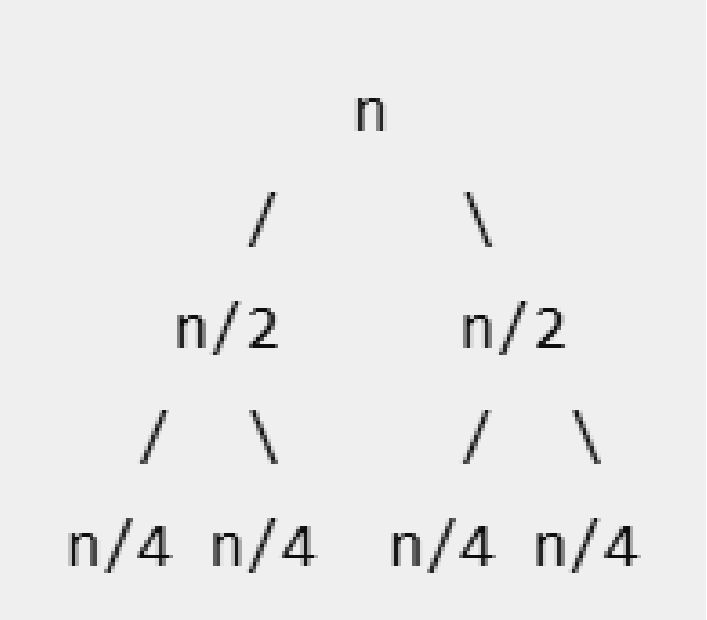
Nível k: $2^k \times \frac{n}{2^k} = n$

Cada nível custa n

Ibmec
PROTAGONISTAS PARA O MUNDO

Exemplo 3 — ÁRVORE DE RECURSÃO

Quantos níveis existem?



$$n \to \frac{n}{2} \to \frac{n}{4} \to \cdots \to 1$$

Depois de **k** níveis: $\frac{n}{2^k} = 1$

$$n = 2^k$$

$$k = \log_2 n$$

Ibmec
PROTAGONISTAS PARA O MUNDO

Exemplo 3 — ÁRVORE DE RECURSÃO

vamos enxergar a recorrência

Nível 0: $1 \times n = n$

Nível 1: $2 \times \frac{n}{2} = n$

Nível 2: $4 \times \frac{n}{4} = n$

Nível 3: $8 \times \frac{n}{8} = n$

Nível k: $2^k \times \frac{n}{2^k} = n$

Cada nível: $\mathbf{n}$

Quantidade de níveis: **k =** $\boldsymbol{log_2\, n}$

Então: $\boldsymbol{T(n) = n \log_2 n}$

Logo: $\boldsymbol{T(n) = \theta(n \log_2 n)}$

Ibmec
PROTAGONISTAS PARA O MUNDO

Exemplo 3: agora precisamos PROVAR

A árvore nos permite formular uma hipótese:

$$T(n) = \theta(n \log_2 n)$$

Mas visualizar não é provar.

Como podemos demonstrar matematicamente?

É aqui que entra o: **Método da Substituição**

Duas etapas:

- formular uma hipótese;
- provar a hipótese.

Ibmec
PROTAGONISTAS PARA O MUNDO

Método da substituição

$$T(n) \leq cn \log n$$

Precisamos provar:

$O(n \log n)$

$\Omega(n \log n)$

Para concluir: $\Theta(n \log n)$

Portanto: $O + \Omega = \Theta$

Ibmec
PROTAGONISTAS PARA O MUNDO

Método da substituição

$$T(n) = 2T(n/2) + n$$

Substituímos $T(n/2)$ pela nossa hipótese:

$$T(n/2) \leq c\frac{n}{2}\log\frac{n}{2}$$

Então: $$T(n) \leq 2\left(c\frac{n}{2}\log\frac{n}{2}\right) + n$$

Ibmec
PROTAGONISTAS PARA O MUNDO

Método da substituição

Então: $$T(n) \leq 2\left(c\frac{n}{2}\log\frac{n}{2}\right) + n$$

Simplificando: $$T(n) \leq cn\log\frac{n}{2} + n$$

Como: $$\log\frac{n}{2} = \log n - 1$$

temos: $$T(n) \leq cn(\log n - 1) + n$$

$$T(n) \leq cn\log n - cn + n$$

Ibmec
PROTAGONISTAS PARA O MUNDO

Método da substituição

Para: $c \geq 1$

$$T(n) \leq cn \log n - cn + n$$

Temos: $-cn + n \leq 0$

Logo: $T(n) \leq cn \log n$

Portanto: $T(n) = O(n \log n)$

Ibmec
PROTAGONISTAS PARA O MUNDO

Método da substituição

Para prova formal completa de: $T(n) = \theta(n \log n)$

Mas é necessário demonstrar também $\Omega(n \log n)$

matematicamente mais correto do que simplesmente concluir θ a partir da prova de O

Ibmec
PROTAGONISTAS PARA O MUNDO

MÉTODO MESTRE

$$T(n) = aT(n/b) + f(n)$$

- a: número de subproblemas;
- n/b : tamanho de cada subproblema;
- $f(n)$: trabalho fora da recursão.

Ibmec
PROTAGONISTAS PARA O MUNDO

Exemplo 3 — usando teorema mestre

```python
def processo(n):
    if n <= 1:
        return

    processo(n // 2)
    processo(n // 2)

    for i in range(n):
        print(i)
```

- a: número de subproblemas;
- n/b : tamanho de cada subproblema;
- $f(n)$: trabalho fora da recursão.

$$T(n) = aT(n/b) + f(n)$$

- 2: número de subproblemas;
- $n/2$: tamanho de cada subproblema;
- $f(n) = n$: trabalho fora da recursão.

$$T(n) = 2T(n/2) + n$$

Ibmec
PROTAGONISTAS PARA O MUNDO

MÉTODO MESTRE: Os três casos

Caso 1: $\boldsymbol{f(n) < n^{\log_b a}}$ $T(n) = \Theta\left(n^{\log_b a}\right)$

Caso 2: $\boldsymbol{f(n) = n^{\log_b a}}$ $T(n) = \Theta\left(n^{\log_b a} \log n\right)$

Caso 3: $\boldsymbol{f(n) > n^{\log_b a}}$ $T(n) = \Theta\left(f(n)\right)$

Essas comparações são uma forma simplificada de apresentar os casos;
o Caso 1 e o Caso 3 possuem condições técnicas adicionais

Ibmec
PROTAGONISTAS PARA O MUNDO

EXEMPLOS DO MÉTODO MESTRE

Caso 1 $\boldsymbol{f(n) < n^{\log_b a}}$ $T(n) = \Theta\left(n^{\log_b a}\right)$

$$T(n) = 4T(n/2) + n$$

- $\boldsymbol{a}$=4
- $\boldsymbol{b = 2}$
- $\boldsymbol{f(n) = n}$

$$n^{\log_2 4} = n^2$$

Como: $n < n^2$ $T(n) = \Theta(n^2)$

Ibmec
PROTAGONISTAS PARA O MUNDO

EXEMPLOS DO MÉTODO MESTRE

Caso 2 $\boldsymbol{f(n) = n^{\log_b a}}$ $T(n) = \Theta\left(n^{\log_b a} \log n\right)$

$$T(n) = 2T(n/2) + n$$

- $\boldsymbol{a}$=2
- $\boldsymbol{b = 2}$
- $\boldsymbol{f(n) = n}$

$$n^{\log_2 2} = n$$

Como: $n = n$ $T(n) = \Theta(n \log n)$

Ibmec
PROTAGONISTAS PARA O MUNDO

EXEMPLOS DO MÉTODO MESTRE

Caso 3 $\boldsymbol{f(n) > n^{\log_b a}}$ $T(n) = \Theta\big(f(n)\big)$

$$T(n) = 2T(n/2) + n^2$$

- $\boldsymbol{a}$=2
- $\boldsymbol{b = 2}$
- $\boldsymbol{f(n) = n^2}$

$n^{\log_2 2} = n$

Como: $n^2 > n$ $T(n) = \Theta(\boldsymbol{n})$

Ibmec
PROTAGONISTAS PARA O MUNDO

Cuidado: nem toda recorrência usa Mestre

Exemplo $\boldsymbol{T(n) = T(n-1) + n}$

Não possui a forma: $\boldsymbol{aT(n/b) + f(n)}$

porque temos: $\boldsymbol{T(n-1)}$ e não: $\boldsymbol{T(n/b)}$

Não podemos aplicar diretamente o Método Mestre.

Neste caso, podemos usar: **Iteração**

Ibmec
PROTAGONISTAS PARA O MUNDO

Comparação dos métodos: Qual método devo usar?

Recorrência	Método
$T(n) = T(n-1) + c$	Iteração
$T(n) = T(n-1) + n$	Iteração
$T(n) = T(n/2) + c$	Iteração
$T(n) = 2T(n/2) + n$	Árvore / Mestre
$T(n) = 4T(n/2) + n$	Mestre
Necessidade de demonstrar hipótese	Substituição

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 1

Para cada função, responda: Quantas chamadas? Qual tamanho? Qual trabalho local? Qual recorrência? Qual método? Qual complexidade?

```
def A(n):
    if n <= 1:
        return
    A(n-1)
```

```
def B(n):
    if n <= 1:
        return
    B(n//2)
```

```
def C(n):
    if n <= 1:
        return
    C(n//2)
    C(n//2)
```

```
def D(n):
    if n <= 1:
        return

    D(n//2)
    D(n//2)

    for i in range(n):
        print(i)
```

Praticando 2

a) Quantas chamadas recursivas existem?

b) Qual o tamanho do subproblema?

c) Qual o custo do trabalho local?

d) Construa a recorrência.

e) Resolva por iteração.

f) Determine Θ

```
def misterio(n):
    if n <= 1:
        return

    for i in range(n):
        print(i)

    misterio(n // 2)
```

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 3

a) Quantas chamadas recursivas existem?
b) Os subproblemas têm o mesmo tamanho?
c) A recorrência possui a forma do Método Mestre?
d) Escreva a recorrência.
e) Qual método você utilizaria? Justifique.

```
def processo(n):
    if n <= 1:
        return

    processo(n // 2)
    processo(n // 4)

    for i in range(n):
        print(i)
```

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 4

Considere: $\boldsymbol{T(n) = 2T(n/2) + 1}$

a) Desenhe os quatro primeiros níveis.

b) Quantos nós existem no nível 0?

c) No nível 1?

d) No nível 2?

e) Qual o custo total de cada nível?

f) Quantos níveis existem?

g) Determine T(n).

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 5

Algoritmo A: $\boldsymbol{T_A(n) = 2T_A(n/2) + n}$

Algoritmo B: $\boldsymbol{T_B(n) = 2T_B(n/2) + 1}$

a) Qual parece ser mais eficiente?
b) Qual é a complexidade de cada um?
c) Por que os dois possuem a mesma quantidade de chamadas recursivas, mas complexidades diferentes?
d) O que representa o termo f(n)?

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 6

a) Quantas chamadas recursivas?

b) Qual o custo do primeiro ***for***?

c) Qual o custo do segundo ***for***?

d) Qual o trabalho local total?

e) Construa a recorrência.

f) Resolva pelo Método Mestre.

```
def exemplo(n):
    if n <= 1:
        return

    exemplo(n // 2)

    for i in range(n):
        print(i)

    for j in range(n):
        print(j)
```

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 7

a) Quantas chamadas recursivas existem em cada chamada?
b) Quais são os tamanhos dos subproblemas?
c) Escreva uma recorrência aproximada para o número de chamadas.
d) Explique por que o Método Mestre não pode ser aplicado diretamente.
e) A árvore de recursão cresce rapidamente? Explique.

```
def fib(n):
    if n <= 1:
        return n


    return fib(n - 1) + fib(n - 2)
```

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 8

a) Qual a recorrência do tempo?

b) Qual a complexidade de tempo?

c) Quantas chamadas ficam simultaneamente na pilha?

d) Qual a complexidade de espaço?

```
def conta(n):
    if n <= 1:
        return

    conta(n - 1)
```

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 9

Compare o exercício 8 com esse:

```
def conta(n):
    if n <= 1:
        return

    conta(n // 2)
```

Ibmec
PROTAGONISTAS PARA O MUNDO

Praticando 10

Considere: $T(n) = 2T(n/2) + n$

Um aluno afirma:
"Como existem duas chamadas recursivas e cada uma recebe metade do problema, então a complexidade é O(n)."

Você concorda? Explique.

Praticando 11

Qual algoritmo é melhor?

$$A(n) = T(n-1) + 1$$

$$B(n) = T(n/2) + n$$

$$C(n) = 2T(n/2) + n$$

a) Determine a complexidade de cada uma.

b) Ordene os algoritmos do mais eficiente para o menos eficiente.

Ibmec
PROTAGONISTAS PARA O MUNDO

Referencias bibliográficas

- SERPA, Matheus S.; RODRIGUES, Thiago N.; ALVES, Ítalo C.; et al Análise de Algoritmos 1. ed. Porto Alegre: SAGAH 2021
- CORMEN, Thomas H.; LEISERSON, Charles E.; Ronald L. Rivest; et al. Algoritmos 4. ed. Rio de Janeiro: GEN LTC2024
- RODRIGUES, Thiago N.; SILVA, Cristiane; HORTA, Gustavo L.; et alTeoria dos grafos e análise de algoritmos 1 ed. Porto Alegre: SAGAH 2022



www.ibmec.br

/ibmec

ibmec

@ibmec_oficial

ibmec

Ibmec

PROTAGONISTAS PARA O MUNDO